

DUOC UC
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

DOCUMENTO DE IMPLEMENTACIÓN DE AMBIENTE EN PRODUCCIÓN

Sistema de Gestión de Reclutamiento Inteligente

"KORELY"

Documento de Implementación de Ambiente Cloud (Railway)
2026

Tabla de Contenido

Datos del Documento e Histórico de Revisiones	3
Información del Proyecto e Integrantes	3
Introducción y Alcance	4
Importancia de un Entorno de Producción Configurado	4
Requisitos del Sistema (Hardware, Conectividad y Software Cloud)	4
Paso 1: Creación y Configuración de Base de Datos Vectorial (PostgreSQL + pgvector)	5
Paso 2: Despliegue de la Capa de Servicios Backend (FastAPI)	5
Paso 3: Despliegue de la Capa de Interfaz Frontend (Next.js)	6
Paso 4: Inicialización, Migraciones y Carga de Datos Semilla (Seed Data)	6
Estructura del Proyecto y Credenciales del Sistema	7
Consideraciones Técnicas y Visualización de Interfaces Validadas	7
Conclusión	8

Datos del Documento

Histórico de Revisiones

Versión	Fecha	Descripción / Cambio	Autor
1.0	20/06/2026	Inicio de documento de implementación de ambiente en la nube (Railway)	Vicente Farias

Información del Proyecto

Organización	Duoc UC. Escuela de Informática y Telecomunicaciones
Proyecto (Nombre)	KORELY
Fecha de Inicio	13/04/2026
Fecha de Término	29/06/2026
Patrocinador Principal	Marcela Gutiérrez (Cipress) / Cristian Gómez (Triskeledu)

Integrantes

Nombre Completo	Cargo Técnico Asignado
Sergio Aranguiz	Gerente de Proyecto / Líder de Desarrollo Frontend y Gestión
Vicente Farias	Gerente de Proyecto / Líder de Desarrollo Backend e IA

Queda prohibido cualquier tipo de explotación y, en particular, la reproducción, distribución, comunicación pública y/o transformación, total o parcial, por cualquier medio, de este documento sin el previo consentimiento expreso y por escrito del equipo de desarrollo de Korely y las organizaciones Cipress / Triskeledu.

Introducción

El presente documento tiene como propósito definir y estandarizar el flujo de implementación y despliegue del ecosistema web del proyecto **Korely** en un entorno productivo basado en la nube. Se detallan de forma minuciosa los prerequisites de infraestructura, las configuraciones de variables de entorno seguras y la secuencia cronológica de pasos requeridos para garantizar la correcta puesta en marcha del sistema.

Esta guía metodológica busca mitigar fallos transaccionales en producción, optimizar el rendimiento del motor de IA y asegurar la consistencia de la arquitectura de microservicios desacoplada en infraestructuras modernas de computación Cloud.

Alcance

Este manual corresponde exclusivamente al despliegue integral del sistema en la plataforma **Railway**, abarcando el aprovisionamiento de la base de datos PostgreSQL con soporte vectorial nativo, la compilación automatizada del servidor API en Python (FastAPI) y la distribución del portal web del cliente estructurado sobre Next.js.

Importancia de un Entorno de Producción Configurado

Contar con una infraestructura productiva en la nube debidamente orquestada y sincronizada permite asegurar:

- Estabilidad, alta disponibilidad (Uptime del 99.9%) y tolerancia a fallos transaccionales.
- Consistencia y aislamiento absoluto en el procesamiento de datos sensibles y currículums de candidatos.
- Escalabilidad autónoma de los servicios de Inteligencia Artificial sin sobrecargar los servidores locales.
- Reducción crítica de latencia en las respuestas de la entrevista virtual conducida por el LLM.

Requisitos del Sistema

Infraestructura Cloud (Hardware Virtualizado en Railway)

Recurso	Especificación del Entorno Cloud
CPU Virtual (vCPU)	Asignación compartida dinámica con tecnología Nixpacks / Docker Containers.
Memoria RAM Virtual	Mínimo: 512 MB por microservicio. Recomendado: 1 GB o superior para el Backend de IA.
Red y Conectividad	Networking interno privado de baja latencia inter-contenedores. Acceso HTTPS público con TLS/SSL.

Software y Tecnologías de Orquestación

El aprovisionamiento automatizado en Railway utiliza especificaciones modernas que prescinden de configuraciones manuales de SO:

- **Plataforma de Despliegue:** Railway App Engine con soporte de compilación automatizada Nixpacks.
- **Base de Datos Relacional Vectorial:** PostgreSQL v15+ con extensión nativa `pgvector` activa.
- **Control de Código y Despliegue Continuo (CI/CD):** Repositorio oficial en GitHub conectado a los webhooks de Railway.

Paso 1: Creación y Configuración de la Base de Datos (PostgreSQL + pgvector)

Railway provee una instancia administrada de PostgreSQL que incluye de forma nativa la librería de tipos vectoriales para la indexación y búsqueda semántica de perfiles mediante similitud coseno.

1. Iniciar sesión en la consola de **Railway (railway.app)**, seleccionar **New Project** y hacer clic en la opción **Provision PostgreSQL**.
2. Una vez completado el aprovisionamiento del contenedor de persistencia, navegar a la pestaña **Variables** para verificar la generación automática de la credencial DATABASE_URL interna del clúster.
3. Conectarse a la instancia de base de datos a través de la consola integrada de Railway o de un cliente externo utilizando la URL de conexión externa provista, y ejecutar la siguiente instrucción estructurada SQL para inicializar el soporte vectorial:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

[Insertar Captura de Pantalla: Consola web de Railway o interfaz de cliente SQL ejecutando exitosamente el comando "CREATE EXTENSION" sobre la base de datos en producción]

Paso 2: Despliegue del Backend (FastAPI)

El motor de análisis de lenguaje natural, parsing de currículums y comunicación con los modelos Gemini se despliega de forma desacoplada siguiendo las siguientes directrices:

1. En el panel del proyecto de Railway, hacer clic en el botón **New**, seleccionar **GitHub Repo** y asociar el repositorio oficial del proyecto.
2. Acceder a la configuración del servicio recién creado y dirigirse a la sección **Settings** para restringir y guiar el build:
 - **Root Directory:** Modificar el valor por defecto a: /Producto/backend
 - **Custom Start Command:** Definir el comando de inicialización de producción de alta concurrencia:

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

3. Ingresar a la pestaña **Variables** e inyectar los accesos y llaves operacionales requeridas por la API:
 - DATABASE_URL: Seleccionar la opción **Add Reference** y enlazar de forma dinámica la variable generada por el contenedor PostgreSQL (`{{Postgres.DATABASE_URL}}`).
 - GEMINI_API_KEY: Digitar la clave de autorización oficial provista por Google AI Studio para el consumo de Gemini Pro.
 - JWT_SECRET_KEY: Cadena de caracteres compleja y encriptada utilizada para la firma de tokens web de sesión.
4. Navegar a **Settings -> Public Networking** y hacer clic en **Generate Domain** para obtener el punto de acceso público de la API.

Paso 3: Despliegue del Frontend (Next.js)

El portal de cara al usuario final y la consola Kanban del reclutador se compilan bajo el framework Next.js utilizando el mismo origen de código:

1. Nuevamente en el panel de Railway, seleccionar **New -> GitHub Repo** y enlazar el mismo repositorio.
2. Hacer clic en el nuevo componente gráfico del frontend, acceder a **Settings** y configurar las variables de entorno de compilación:
 - **Root Directory:** Modificar el valor por defecto a: `/Producto/Frontend`
3. Acceder a la pestaña **Variables** y registrar las variables de entorno públicas consumidas durante el proceso de empaquetado (Build React):
 - `NEXT_PUBLIC_BACKEND_URL`: Pegar la dirección URL pública generada en el Paso 2 para el backend de FastAPI (Omitir la barra diagonal inversa `/` al final).
 - `NEXT_PUBLIC_GEMINI_API_KEY`: Proveer la clave de API de Google Gemini para habilitar el procesamiento multimedia y reconocimiento de audio conversacional del cliente.
4. Dirigirse a **Settings -> Public Networking** y generar el dominio público del portal web. El motor Nixpacks de Railway compilará el código ejecutando los comandos estructurados `npm run build` y `npm run start` de forma automática.

[Insertar Captura de Pantalla: Panel principal de Railway con los tres microservicios (PostgreSQL, Backend y Frontend) activos en color verde, indicando que el despliegue automático ha concluido con éxito]

Paso 4: Inicialización, Migración y Carga de Datos Semilla (Seed Data)

Para aplicar la estructura relacional de las tablas y registrar los perfiles iniciales de administración en la base de datos de producción, se recomienda ejecutar el script de sanidad de forma remota:

1. Navegar a la pestaña **Variables** de la instancia PostgreSQL de Railway y copiar la dirección completa de la variable de conexión externa `DATABASE_URL`.
2. En la estación de trabajo local, ingresar al directorio `/Producto/backend` y abrir de manera temporal el archivo de configuración de entornos local `.env`.
3. Reemplazar momentáneamente el valor de la variable local `DATABASE_URL` por la cadena de conexión externa copiada de la nube de Railway.
4. Ejecutar el script de mapeo relacional en la consola local con el entorno virtual activo:

```
python setup_formal_db.py
```

5. Una vez que la consola confirme la inyección de las cuentas iniciales, revertir los cambios del archivo `.env` local para apuntar nuevamente a la base de datos Docker local (`localhost:5433`).

Estructura del Proyecto

Distribución jerárquica de directorios fundamentales en el repositorio conectado a la nube:

```
Producto/
├── backend/           # Componente de Microservicios (FastAPI)
│   ├── main.py       # Script principal de FastAPI y enrutamiento
│   ├── requirements.txt # Suite de dependencias del servidor de producción
│   └── setup_formal_db.py # Script de inicialización estructural de datos
└── Frontend/         # Interfaz Gráfica del Sistema (Next.js)
    ├── package.json  # Paquetes y scripts de Node.js
    ├── src/          # Código fuente de las vistas y componentes React
    └── public/        # Recursos multimedia y estáticos
```

Credenciales del Sistema en Producción

Posterior a la ejecución del script de inicialización del Paso 4, el acceso al portal en la nube se encuentra validado para las siguientes cuentas de prueba de datos semilla:

- **Rol: Gerente (Reclutador / Administrador):**
 - Correo electrónico: `carlos.valenzuela@cipress.cl`
 - Contraseña de acceso: `password123`
- **Rol: Postulante (Candidato):**
 - Correo electrónico: `esteban.diaz@example.com`
 - Contraseña de acceso: `password123`

Consideraciones Técnicas y Vistas de Interfaz

El entorno web productivo se comunica mediante peticiones HTTP asíncronas y seguras. Se garantiza que todas las transacciones, desde la carga automática del currículum PDF hasta el cálculo del Score Consolidado vectorial en la base de datos `pgvector`, posean un tiempo de respuesta optimizado **inferior a los 5 segundos**.

[Insertar Captura de Pantalla: Interfaz de Acceso/Login adaptativa desplegada desde la URL pública de producción, mostrando el formulario web con los campos de autenticación de Korely AI]

[Insertar Captura de Pantalla: Consola Dashboard de Reclutador de Korely en producción, exponiendo las métricas cuantitativas del volumen de talento, ranking de afinidad semántica y el Pipeline Kanban funcional]

Conclusión

La correcta implementación del entorno en la nube para la plataforma **Korely** permite consolidar una solución de nivel empresarial con un alto estándar de seguridad y eficiencia. La separación arquitectónica

desacoplada asegura que el procesamiento intensivo del parsing NLP de los archivos adjuntos y las consultas vectoriales de alta dimensión con pgvector no afecten la fluidez de la interfaz del usuario en Next.js.

A través del uso estructurado de variables de entorno y el flujo continuo de integración (CI/CD) provisto de manera nativa por la infraestructura de Railway, el ecosistema de reclutamiento inteligente queda completamente automatizado, blindado ante accesos no autorizados y listo para escalar operativamente según el volumen transaccional demandado por el cliente comercial.